

MoE-CORE: Coordinated Expert Offloading and Residency for Memory-Constrained MoE Inference

Ke Yang^{3*}, Yongji Gao^{1,2*}, Xushi Li^{1,2†}, Kui Luo¹, Sicheng Zhang^{1,2}, Tianming Zhou^{1,2}, Keyi Liu¹, Shufang Lu¹, Aoxuan Chen¹, Jie Meng¹, Jingchun Gao^{1,2}, Dan Li^{1,2}, Xinkai You¹, Dan Li¹, Zhixiang Xia^{1,2}, Yan Shi⁴, Yang Liu¹, Yanjia Zeng¹, Liangjun Feng^{1,2}

¹Huawei Technologies Co. Ltd

²Huawei Ascend Computing Technology Development

³University of Chinese Academy of Sciences

⁴Shanghai Jiao Tong University

yangke@iie.ac.cn keyi.liu@h-partners.com amy_sy@sjtu.edu.cn

{gaoyongji2, lixushi, luokui, zhangsicheng5, zhoutianming2, shufang.lu, chenaoxuan1, mengjie48, gaojingchun1, lidan276, youxinkai, xiazhixiang1, yang.liu8, zengyanjia, fengliangjun}@huawei.com

Abstract

Sparse expert activation reduces MoE models’ computation, yet expert weights can exceed limited device memory. Offloading makes inference feasible on a compact AI appliance but exposes host-to-device transfers to the inference path. We present MoE-CORE, a system that coordinates expert offloading and residency for memory-constrained MoE inference. It stages complete expert layers in alternating buffers during prefill. During decode, it combines nonuniform layer-wise cache capacity, domain-informed initialization, routing-history-aware replacement, and cross-layer prefetching. The main configuration executes router-selected experts exactly; an optional score-based substitution path handles eligible low-score misses. The main comparison uses 1K- and 128-token output caps for MoE-CORE and vLLM Prefetch, respectively. Across five workloads per model, MoE-CORE records a mean time per output token (TPOT) of 38.0–44.8 ms versus 1268.9–1269.1 ms for the evaluated vLLM Prefetch configuration on DeepSeek-V4-Flash-W4A8; the corresponding values on GLM-5.2-W4A8C8 are 206.6–220.5 and 5941.5–5941.8 ms. Under an 84-GB NPU-memory cap, the best measured DeepSeek GSM8K configuration achieves a TPOT of 21.5 ms with approximate expert substitution and multi-token prediction (MTP) at depth 2. These results support coordinated expert residency and transfer scheduling under a device-memory constraint. The code is [here](#).

1 Introduction

Mixture-of-Experts (MoE) architectures (Shazeer et al., 2017) have become an important approach to scaling the capacity of large language models (LLMs) without proportionally increasing per-token computation. By routing each token to activate only a small subset of experts, MoE models offer a favorable computation-to-capacity ratio and are therefore attractive for resource-constrained local and edge inference (Yi et al., 2025). Sparse activation, however, reduces computation rather than the amount of model weights that must remain accessible. Recent model families make this distinction increasingly consequential: DeepSeek-V4-Flash and DeepSeek-V4-Pro (DeepSeek-AI, 2026) contain 284B and 1.6T total parameters while activating 13B and 49B parameters per token, respectively; GLM-5.2 (GLM-5 Team, 2026) contains 744B total parameters with 40B active; and Kimi K3 scales to 2.8T total parameters with 104B active (Kimi-Team, 2026). These models also increasingly target Agent workloads such as long-horizon coding, tool use, and knowledge work. Deploying them as local Agent services therefore requires attention not only to sparse computation, but also to the storage capacity and memory cost of the full expert set.

This memory burden is particularly acute on compact, single-node AI appliances. These appliances integrate a host processor and memory with one or a small number of GPUs or NPUs, providing enterprises with a practical on-premises platform for deploying AI services close to private data while retaining local control over the serving en-

*The two authors contributed equally to this work.

†Corresponding author

environment. However, the device-memory capacity of these systems is under increasing pressure from two directions: Growing MoE parameter counts increase the volume of weights, while long-context Agent tasks expand the KV cache with conversation histories, retrieved documents, tool outputs, and intermediate task state. Thus, these demands can prevent the complete set of model weights from residing in device memory, even though an AI appliance provides substantially more resources than a mobile or embedded edge device. Deploying LLMs on such systems therefore requires placing some of the model weights in host memory.

Expert offloading makes such deployments feasible by keeping some routed expert weights in host memory and transferring them to the device memory (GPUs or NPUs) when needed. Because expert weights account for a large fraction of an MoE model’s parameters while each token activates only a small subset of experts, retaining inactive experts in host memory can substantially increase the model size supported by a fixed device-memory budget. This technique reduces device-memory demand, but loading a selected expert after routing places host-to-device (H2D) communication on the inference critical path. Existing techniques for reducing this cost can be grouped into four categories. (i) **Activation-locality methods** use routing traces or semantic history to identify experts that are likely to be reused and retain them in a device cache, as in MoE-Infinity and FineMoE (Xue et al., 2025; Yu et al., 2025). (ii) **Auxiliary-predictor methods** train a separate lightweight model to predict expert activations before the original router is reached. SiDA-MoE (Du et al., 2024) uses an offline-trained hash predictor, while ProMoE trains predictors for individual layers (Song et al., 2025). (iii) **Cross-layer and phase-aware methods** obtain expert information from later gates or reorganize transfers according to layer and inference phase. Fate (Fang et al., 2026b) and AdapMoE (Zhong et al., 2024a) use cross-layer gating with layer-aware expert management, whereas DuoServe-MoE (Zhang et al., 2026) separates prefill and decode transfer pipelines. (iv) **Expert-substitution methods**, including SMOE and BuddyMoE (Zhu et al., 2026; Wang et al., 2025a), avoid selected transfers in eligible cases by executing an already resident alternative expert. Together, these systems show that offloading latency can be addressed through caching, prediction, scheduling, and fallback mechanisms. However, these methods

have been evaluated on short-context workloads, leaving the interaction between long contexts, expert residency, and offloading latency insufficiently characterized.

For memory-constrained MoE inference on a compact AI appliance, however, these mechanisms must operate as parts of a coordinated runtime rather than as isolated policies. This requirement raises three design questions:

⚙️ *Does equal per-layer cache allocation yield comparable expert hit rates across MoE layers?*

📊 *Do expert popularity rankings differ across domain-specific workloads?*

🔍 *Are routing scores near the Top-k boundary close enough to motivate a resident alternative for an eligible low-score cache miss?*

To address these questions, we present MoE-CORE, a coordinated expert-offloading system for memory-constrained MoE inference on compact, single-node AI appliances.

MoE-CORE assigns prefill and decode to different expert-offloading paths and reserves a separate device-memory region for each phase. For prefills, it stages the complete routed-expert weights of two consecutive MoE layers in two alternating buffers, overlapping the transfer of the next layer with computation of the current layer. Decode is the primary optimization target because it activates a sparse expert set at each iteration, yet an uncovered expert transfer can directly delay token generation.

During decoding, MoE-CORE first distributes the expert cache budget across all MoE layers according to a dynamic layer-wise cache allocator. Given a workload-domain hint from the serving application, an offline profile initializes each layer’s cache with the corresponding hot experts. A routing-history-aware online cache then adapts this initial placement to the routing behavior observed during the current session. Because expert residency alone cannot eliminate all misses, a cross-layer prefetcher evaluates the next layer’s router early and overlaps predicted expert transfers with useful computation. For residual misses, high-score experts remain on the exact loading path, whereas only eligible low-score experts may use resident substitutes selected by routing-score proximity.

Relative to Load on Demand (LoD), which allocates static per-layer expert caches only for decode and synchronously loads non-resident selected experts, MoE-CORE achieves mean time per output token (TPOT) speedups of up to $1.75\times$ on

DeepSeek-V4-Flash-W4A8 and $1.15\times$ on GLM-5.2-W4A8C8. MoE-CORE also records lower TPOT than the evaluated vLLM Prefetch configurations. Its best measured configuration achieves 21.5 ms TPOT with multi-token prediction (MTP) at depth 2.

Our contributions are as follows:

- We characterize layer-wise differences in cache coverage and workload-dependent expert rankings under a fixed cache configuration, and examine routing scores near the Top- k activation boundary.
- We design phase-specific expert-offloading paths for prefill and decode within a shared device-memory budget.
- We coordinate layer-aware cache allocation, domain-informed initialization, online cache adaptation, and cross-layer prefetching; optional score-based substitution handles eligible low-score residual misses while high-score misses retain exact loading.
- We demonstrate MoE-CORE’s effectiveness on five workloads with DeepSeek-V4-Flash-W4A8 and GLM-5.2-W4A8C8 on their respective AI-appliance platforms. It records lower TPOT than both LoD and the evaluated vLLM Prefetch configurations, while ablations isolate the effects of its cache-management and prefetching mechanisms.

2 Background and Motivation

2.1 MoE Inference

A Mixture-of-Experts (MoE) model replaces the dense feed-forward network in selected Transformer layers with multiple expert feed-forward networks (FFNs) and a router. For token t in MoE layer l , the router scores all N_l routed experts from the hidden state $h_{l,t}$ and selects the top- k_l :

$$\mathbf{r}_{l,t} = R_l(h_{l,t}), \mathcal{S}_{l,t} = \text{TopK}(\mathbf{r}_{l,t}, k_l), \quad (1)$$

where R_l is the router, $\mathbf{r}_{l,t}$ contains the expert scores, and $\mathcal{S}_{l,t}$ is the set of selected expert indices. The routed MoE output is

$$m_{l,t}^{\text{routed}} = \sum_{e \in \mathcal{S}_{l,t}} \alpha_{l,t,e} E_{l,e}(h_{l,t}), \quad (2)$$

where $E_{l,e}$ is the FFN of expert e , and $\alpha_{l,t,e}$ is its normalized routing weight. Because $\mathcal{S}_{l,t}$ depends

on the hidden state, expert selection can vary across tokens and layers. Each token evaluates only k_l experts, but the runtime must provide access to all N_l expert weights (Shazeer et al., 2017).

Let $s_{l,e}$ denote the stored size of routed expert e in layer l . The complete routed-expert footprint is

$$M_{\text{routed}} = \sum_{l=1}^L \sum_{e=1}^{N_l} s_{l,e}. \quad (3)$$

Thus, routed-expert storage is determined by the total expert set rather than by the number activated for each token. Routed expert weights consequently account for a large fraction of the model state that must be stored somewhere in the serving system.

2.2 Expert Offloading on Heterogeneous Memory

Compact AI appliances provide a heterogeneous memory hierarchy. GPU or NPU device memory offers high bandwidth for model execution but limited capacity, whereas host memory provides greater capacity at lower bandwidth. Expert offloading exploits the sparsity of MoE routing by keeping only a subset of experts resident in device memory and retaining the remaining experts in host memory (Xue et al., 2025; Yi et al., 2025). When the router selects a non-resident expert, its weights must be transferred to the device before execution.

For a single device, expert residency must satisfy

$$M_{\text{fixed}} + M_{\text{KV}}(B, T) + M_{\text{act}}(B, T) + M_{\text{workspace}} + \sum_{l=1}^L \sum_{e=1}^{N_l} x_{l,e} s_{l,e} \leq M_{\text{device}}, \quad (4)$$

where $x_{l,e} \in \{0, 1\}$ indicates whether expert e in layer l is resident, B is the batch size, T is the context length, and M_{fixed} denotes non-routed-expert weights and other persistent state.

Offloading improves memory feasibility, but a cache miss introduces host-to-device (H2D) communication. For an expert of size $s_{l,e}$, its transfer time can be approximated as

$$t_{l,e}^{\text{H2D}} \approx \tau_{\text{launch}} + \frac{s_{l,e}}{\beta_{\text{H2D}}}, \quad (5)$$

where β_{H2D} is the effective H2D bandwidth and τ_{launch} is the fixed transfer overhead. If the transfer begins only after experts are selected and cannot overlap with computation, its time lies on the inference critical path. Expert-offloading performance therefore depends on both the resident expert set and the ability to hide remaining transfers.

2.3 Inference Phases and Memory Pressure

Long-context Agent workloads illustrate the memory pressure of MoE inference: multi-step tasks accumulate conversation turns, retrieved evidence, tool outputs, and intermediate task state. Agent-LongBench (Fang et al., 2026a) models such workloads with trajectories ranging from 32K to 4M tokens, spanning long-turn interactions and high-information-density tool responses.

MoE inference follows a prefill–decode workflow. Prefill processes the input context, typically in parallel, and constructs the KV cache; its latency is a major component of time to first token (TTFT). Decode reuses the cache to generate subsequent tokens iteratively, with time per output token (TPOT) characterizing its per-token latency (Agrawal et al., 2024; Zhong et al., 2024b). In MoE models, prefill may access a broad expert set across many context tokens, whereas each decode step routes only the newly generated token or token group.

Longer trajectories expand the KV cache, leaving less device memory for resident experts. Context length and KV-cache representation therefore jointly determine the expert-residency budget.

2.4 Observation and Motivation

Memory constraints limit how many experts can remain resident on the device, while routing determines which experts are needed during inference. The resulting offloading cost depends on the interaction between expert residency and routing decisions, particularly when a selected expert is absent from device and its transfer cannot be hidden.

Equal Per-Layer Cache Sizes Lead to Uneven Hit Rates. Figure 1 compares five workloads with 84 resident experts per DeepSeek MoE layer, excluding zero-based layers 0–2, whose routing differs. Across layers 3–42, each workload’s hit-rate range spans 7.64–12.07 percentage points. Layers 4 and 7 lie in the bottom ten in all five workloads; layers 7, 9, 19, 25, 27, 31, and 37 are strict local troughs in every trace (below both neighbors). Thus, a uniform per-layer quota yields recurrent coverage deficits at specific layers across the measured workloads. This motivates dynamic layer-wise allocation of a fixed expert-cache budget to improve coverage where additional capacity is most effective (Section 3.1).

Expert Routing May Exhibit Domain-Specific Bias. Figure 2 shows a representative comparison of five workloads at MoE layer 20. Their top-

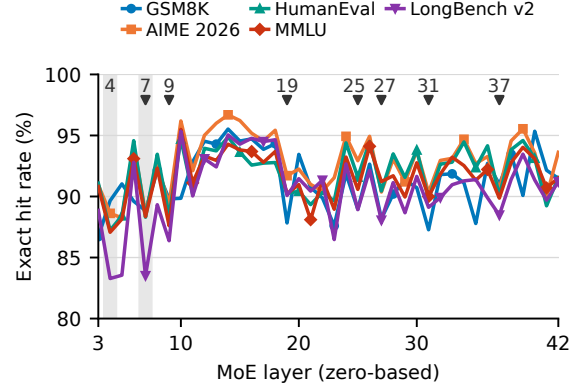


Figure 1: Exact selected-expert hit rate on DeepSeek-V4-Flash-W4A8 with 84 cached experts per layer. The y-axis spans 80–100%. Gray bands mark layers 4 and 7, both bottom-ten in all five datasets. Triangles mark layers 7, 9, 19, 25, 27, 31, and 37, strict local troughs (below both neighbors) in all five.

ranked expert IDs differ: 21 for GSM8K, 179 for AIME 2026, 1 for HumanEval, 172 for MMLU, and 235 for LongBench v2. Across the ten workload pairs at this layer, Spearman correlations over all 256 expert scores range from 0.546 to 0.784, indicating shared ranking structure. Yet Top-6 Jaccard overlap ranges from 0 to 0.200 (mean 0.116), and Top-32 overlap ranges from 0.306 to 0.455. Across the 40 MoE layers (3–42), the top-ranked expert is not identical across workloads in 37 layers, with a mean pairwise Top-6 Jaccard overlap of 0.184. This recurring divergence indicates domain-conditioned routing bias and motivates domain-informed cache initialization.

Near-Boundary Scores Motivate Resident Expert Substitution. We examine normalized routing scores across DeepSeek MoE layers 3–42 and zoom in on layer 20 (Figure 3). Panel (a) shows only a small group of high-scoring experts per layer; most experts occupy the low-score region. Panel (b) shows that the activation boundary can have a narrow score gap: the last activated expert (rank 6, ID 195) scores 2.5010%, while the first non-activated expert (rank 7, ID 231) scores 2.4493%, a difference of 0.0517 percentage points. Thus, expert selection need not imply a large score separation at the Top-6 cutoff. This motivates substituting a similar-scoring resident non-activated expert for an off-device low-score activated expert to bypass H2D transfer.

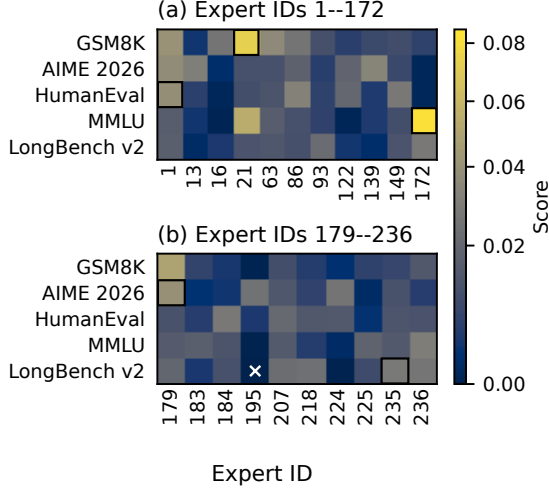


Figure 2: Normalized routing scores for DeepSeek-V4-Flash-W4A8 at layer 20. The union of the five Top-6 expert sets (21 IDs) is ordered by expert ID and split into two blocks. Black outlines mark each workload’s top expert; the cross marks an omitted zero-score ID.

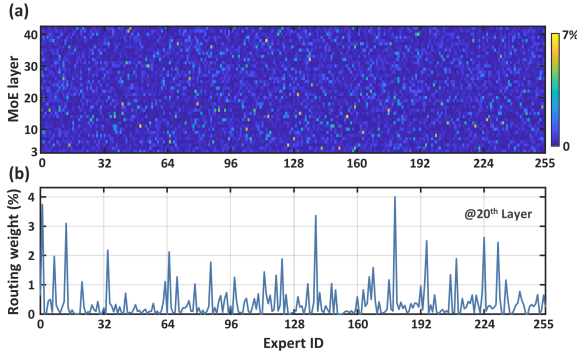


Figure 3: Illustrative DeepSeek-V4-Flash-W4A8 routing scores on AIME 2026. (a) Score distributions across MoE layers 3–42. (b) Scores by expert ID at layer 20, including the rank-6/rank-7 Top-6 boundary.

3 MoE-CORE

MoE-CORE is a coordinated expert-weight offloading system for memory-constrained MoE inference on an AI appliance. Its design follows the phase asymmetry described in Section 2.4: prefill can collectively access a broad expert set, whereas each decode iteration exposes a sparse but latency-sensitive working set. Applying one offloading policy to both phases would either transfer unnecessary experts during decode or incur repeated expert-selection and loading overhead during long prefills. MoE-CORE therefore provides separate prefill and decode paths within a unified runtime; Figure 4 shows the process.

The two paths use physically disjoint device-

memory regions that remain allocated throughout inference. The prefill region contains two expert-weight buffers, each holding all routed experts from a single MoE layer. When the prefill token count reaches the cache-dependent threshold N_l^* defined in Section 3.6, the buffers stage the expert weights of two consecutive layers and alternate as execution advances through the model. The decode region stores routed-expert weights across all MoE layers, organizing its capacity into per-layer caches with K_l resident expert slots assigned to layer l . Both regions share the device-memory budget with the fixed model state, KV cache, activations, and runtime workspace.

During decode, MoE-CORE first establishes expert residency before attempting to hide communication. A layer-wise allocator determines K_l . The serving application then specifies the workload domain through a request hint, and MoE-CORE fills the allocated slots with the corresponding hot experts identified by offline profiling. As decoding proceeds, a Routing-History-Aware Cache (RHAC) policy updates expert residency using recent activations, longer-term routing history, router scores, and access recency. These mechanisms reduce cache misses but cannot eliminate them. MoE-CORE then adopts a cross-layer prefetcher that predicts the next layer’s routed experts and asynchronously transfers predicted experts during the intervening computation. If a selected expert remains unavailable when required, the exact path loads it from host memory. When optional approximate expert substitution is enabled, an eligible low-score miss may instead use a score-similar, unselected expert already resident on the device; high-score misses remain on the exact path.

Thus, the decode path combines miss reduction with communication overlap before considering approximate execution. Layer-wise allocation, domain-profile initialization, RHAC, and cross-layer prefetching preserve the routed expert computation; only optional expert substitution changes the selected expert set.

3.1 Dynamic Layer-Wise Expert-Cache Allocation

The decode expert region has a fixed memory capacity, but that capacity need not be divided equally among MoE layers. Let M_{dec} be the memory assigned to the decode expert cache, K_l the number of expert slots allocated to layer l , and S_l the size of one expert in that layer. The allocation must

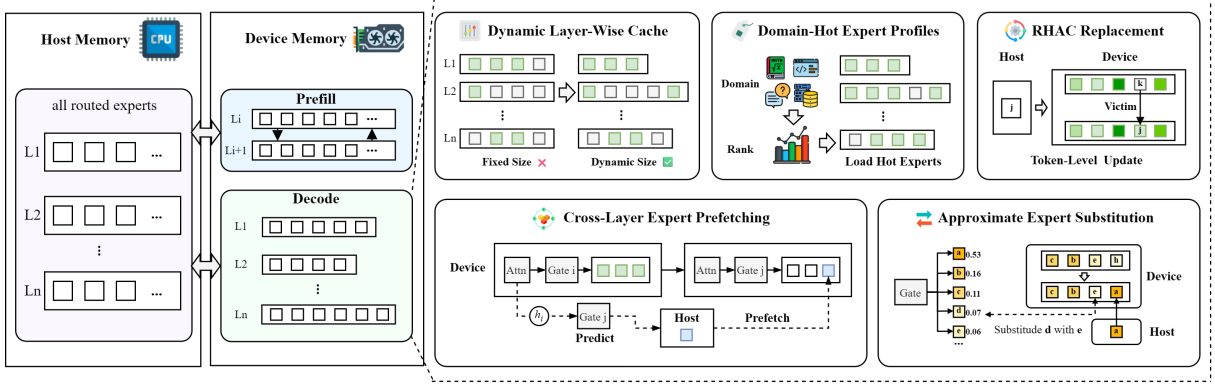


Figure 4: MoE-CORE’s phase-specific expert-weight paths and decode-cache mechanisms. The prefill buffers stage complete MoE layers, whereas the decode region retains per-layer expert subsets and supports domain-based initialization, RHAC replacement, cross-layer prefetching, and optional substitution.

satisfy $\sum_{l=1}^L K_l S_l \leq M_{\text{dec}}$.

A uniform design assigns the same number of slots to every layer. Section 2.4 identifies low-hit layers that recur across the measured workloads, making them candidates for additional slots. A low hit rate does not by itself establish that more capacity will help, so MoE-CORE profiles each layer’s response to cache-size changes before redistributing the decode expert cache.

The allocator first replays representative routing traces and measures the layer-wise hit-rate curve $\hat{H}_l(k)$ for a small set of candidate cache sizes around the uniform allocation. The replacement and initialization policies are held constant during this profiling so that the measured difference reflects only the number of slots. Starting from the uniform allocation, the allocator moves slots in increments of ΔK . For the current allocation K_l , it estimates the loss caused by removing ΔK slots and the gain obtained by adding them as

$$\begin{aligned} D_l^- &= \hat{H}_l(K_l) - \hat{H}_l(K_l - \Delta K), \\ G_l^+ &= \hat{H}_l(K_l + \Delta K) - \hat{H}_l(K_l). \end{aligned} \quad (6)$$

At each allocation round, layers whose hit rates remain above the layer-wise median and whose removal loss is no greater than ϵ_{drop} form the donor set. Layers whose hit rates fall at least τ_{low} below the median form the receiver set. MoE-CORE selects the donor d with the smallest D_d^- and the receiver r with the largest G_r^+ . If $G_r^+ > D_d^-$ and the resulting allocation satisfies the memory constraint, it updates

$$K_d \leftarrow K_d - \Delta K, \quad K_r \leftarrow K_r + \Delta K. \quad (7)$$

The donor and receiver sets are recomputed after every transfer. Reallocation stops when no receiver remains below the low-hit threshold, no donor can release slots within the permitted drop, or no feasible transfer offers positive estimated gain. The resulting allocation $\{K_l\}_{l=1}^L$ remains within the decode-cache budget while shifting capacity from locally insensitive high-hit layers to cache-sensitive low-hit layers.

Profiling and allocation run outside the per-token inference path and are repeated when the model, deployment workload, or available decode-cache capacity changes. The allocator determines only how many expert slots each layer receives. The selected workload-domain profile determines which experts initially fill those slots, and RHAC subsequently updates their contents during decoding. Thus, layer-wise allocation changes neither the router decisions nor the model computation.

3.2 Offline Domain Hot-Expert Profiles

Agent workloads span distinct application domains, including software engineering, mathematical reasoning, and domain-specific tool use (Jimenez et al., 2024; Gou et al., 2024; Yao et al., 2024). The domain-dependent expert rankings observed in Section 2.4 motivate domain-conditioned cache initialization. For each supported domain, MoE-CORE constructs an offline hot-expert profile. At request admission, the serving scheduler supplies a domain identifier d , and the corresponding profile determines which experts initially occupy the allocated cache slots.

We construct these profiles for a fixed set of commonly served domains using mathematical-reasoning, code-generation, and QA datasets. For

domain d , let $\mathcal{T}_d$ denote all tokens in its profiling dataset, and let $\mathcal{E}_l = \{1, \dots, N_l\}$ be the complete expert set at layer l . For every token $t \in \mathcal{T}_d$, the profiler records the selection score $r_{l,t,e}$, the e -th component of $\mathbf{r}_{l,t}$, for every expert $e \in \mathcal{E}_l$ before sparse expert selection. The mean selection score of expert e is

$$\mu_{d,l,e} = \frac{1}{|\mathcal{T}_d|} \sum_{t \in \mathcal{T}_d} r_{l,t,e}. \quad (8)$$

The vector $\boldsymbol{\mu}_{d,l} = (\mu_{d,l,1}, \dots, \mu_{d,l,N_l})$ summarizes the domain-conditioned selection preference over all experts in layer l . For every domain-layer pair, the profiler stores a permutation $\pi_{d,l}$ satisfying

$$\mu_{d,l,\pi_{d,l}(1)} \geq \mu_{d,l,\pi_{d,l}(2)} \geq \dots \geq \mu_{d,l,\pi_{d,l}(N_l)}. \quad (9)$$

Once the layer-wise allocator assigns K_l cache slots to layer l , MoE-CORE initializes the resident set with the K_l experts having the highest mean selection scores:

$$\mathcal{R}_l^{\text{init}}(d) = \{\pi_{d,l}(1), \dots, \pi_{d,l}(K_l)\}. \quad (10)$$

This construction separates cache capacity from cache contents: the layer-wise allocator determines how many experts layer l can retain, while the selected domain profile determines which experts initially occupy those slots.

3.3 Routing-History-Aware Expert Cache

The offline layer-wise allocation and domain profiles cannot capture routing changes within a session. MoE-CORE therefore manages each layer's resident experts with RHAC. Like LRU, RHAC selects eviction victims, but it bases the decision on both activation history and the router scores already produced during MoE inference.

RHAC is updated per token. Let $i = 1, 2, \dots$ denote the update order, let t_i be the token processed at update i , and let $W \in \mathbb{N}^+$ denote the fixed history-window length in tokens. RHAC maintains four signals for every expert e at layer l : recent selection frequency f , long-term selection tendency $\bar{f}$, the current router score q , and age a . Before the first generated token, the recent-frequency window is empty and the two recurrent states are initialized as $\bar{f}_{l,e}^{(0)} = a_{l,e}^{(0)} = 0$.

First, the recent selection frequency counts how many of the latest W tokens select expert e :

$$f_{l,e}^{(i)} = \sum_{j=\max(1, i-W+1)}^i \mathbb{I}[e \in \mathcal{S}_{l,t_j}]. \quad (11)$$

Thus, $f_{l,e}^{(i)}$ ranges from zero to $\min(i, W)$. Second, RHAC captures a longer-term selection tendency with an exponential moving average (EMA) (Brown, 1956),

$$\bar{f}_{l,e}^{(i)} = \rho_{\text{sel}} \bar{f}_{l,e}^{(i-1)} + (1 - \rho_{\text{sel}}) \mathbb{I}[e \in \mathcal{S}_{l,t_i}]. \quad (12)$$

Here, $0 \leq \rho_{\text{sel}} < 1$ is the selection-history decay factor. Values closer to one retain earlier selections for longer, whereas values closer to zero emphasize the current token. Third, the current router scores provide information about every expert. Let $q_{l,t,e} = r_{l,t,e}$ denote the expert-selection score assigned to expert e for token t . RHAC uses the current value $q_{l,t_i,e}$ directly. Finally, age records how many generated tokens have elapsed since the expert was last selected:

$$a_{l,e}^{(i)} = \begin{cases} 0, & e \in \mathcal{S}_{l,t_i}, \\ a_{l,e}^{(i-1)} + 1, & e \notin \mathcal{S}_{l,t_i}. \end{cases} \quad (13)$$

Because the four signals have different scales, each is normalized across all N_l experts in the same layer before combination; a tilde denotes the resulting layer-wise normalized value. RHAC assigns expert e the residency-priority score

$$\eta_{l,e}^{(i)} = \lambda_f \tilde{f}_{l,e}^{(i)} + \lambda_m \tilde{\bar{f}}_{l,e}^{(i)} + \lambda_q \tilde{q}_{l,t_i,e} - \lambda_a \tilde{a}_{l,e}^{(i)}, \quad (14)$$

where the four nonnegative coefficients λ_f , λ_m , λ_q , and λ_a control the contributions of recent frequency, the selection EMA, the current router score, and age, respectively. Recent and sustained selections raise the priority, a high router score protects an expert that is relevant to the present token, and increasing age lowers the priority of stale experts.

When a routed miss must enter a full cache, RHAC selects the coldest evictable resident expert,

$$e_{l,i}^{\text{victim}} = \arg \min_{e \in \mathcal{R}_l \setminus \mathcal{P}_{l,i}} \eta_{l,e}^{(i)}, \quad (15)$$

where $\mathcal{R}_l$ is the current resident set and $\mathcal{P}_{l,i}$ contains experts that are executing, loading, or otherwise protected from eviction while token t_i is processed. After the victim is released and the requested weights arrive, the new expert occupies the slot. Therefore, RHAC remains free to adapt the cache to the observed session.

3.4 Cross-Layer Expert Prefetching

Even RHAC cannot eliminate every decode expert miss. Loading the missed expert only after the true

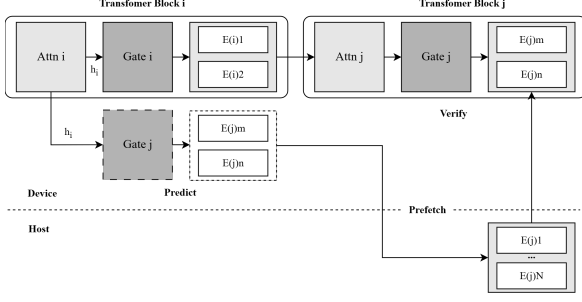


Figure 5: Cross-layer expert prefetching. The next layer’s router receives the current layer’s hidden state to rank candidate experts before its ordinary input becomes available; predicted experts are transferred during intervening computation.

router finishes places the complete H2D transfer on the critical path. MoE-CORE predicts the next layer’s expert ranking early and transfers a bounded number of likely experts while the model continues computing, as illustrated in Figure 5.

Section 2.1 defines $h_{l,t}$ as the hidden state used for routing token t at layer l . Before the normal router input $h_{l+1,t}$ becomes available, MoE-CORE evaluates the original router of layer $l + 1$ on $h_{l,t}$:

$$\hat{\mathbf{r}}_{l+1,t} = R_{l+1}(h_{l,t}). \quad (16)$$

Here, $\hat{\mathbf{r}}_{l+1,t}$ is the predicted expert-score vector for layer $l + 1$, and MoE-CORE prefetches experts according to its ranking. This prediction is motivated by cross-layer routing consistency: routing decisions in adjacent MoE layers can remain correlated, allowing $R_{l+1}(h_{l,t})$ to approximate the ranking later produced by $R_{l+1}(h_{l+1,t}) = \mathbf{r}_{l+1,t}$ (Fang et al., 2026b; Zhong et al., 2024a).

The prefetch count is determined by the computation window available for hiding communication. Let

$$T_l^{\text{overlap}} = T_l^{\text{remaining}} + T_{l+1}^{\text{attn}} \quad (17)$$

be the interval from the early prediction until the true layer- $l + 1$ gate completes. It contains the remaining computation of layer l and the attention computation of layer $l + 1$. If the effective H2D bandwidth is β_{H2D} , the transfer launch overhead is τ_{H2D} , and each layer- $l + 1$ expert occupies s_{l+1} bytes, an upper bound on the number of complete expert transfers hidden by this interval is

$$n_{l+1}^{\text{hide}} = \left\lfloor \frac{\max(0, T_l^{\text{overlap}} - \tau_{\text{H2D}}) \beta_{\text{H2D}}}{s_{l+1}} \right\rfloor. \quad (18)$$

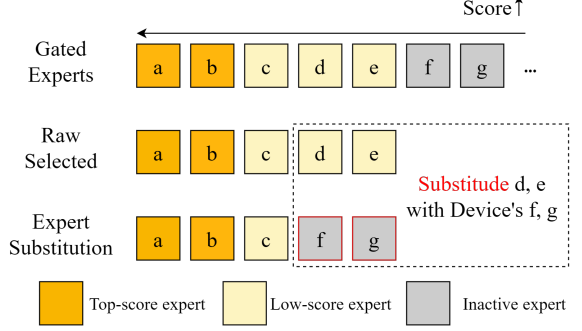


Figure 6: Illustrative expert substitution near the Top- k boundary. Selected experts d and e are off-device, while lower-ranked unselected experts f and g are resident. Replacement is considered only when their routing scores satisfy the proximity criterion.

Let n_{l+1}^{cand} be the number of predicted experts that are neither resident nor already loading, and let n_{l+1}^{slot} be the number of slots the runtime can make available without evicting an in-use expert. MoE-CORE prefetches

$$n_{l+1}^{\text{pf}} = \min(n_{l+1}^{\text{hide}}, n_{l+1}^{\text{cand}}, n_{l+1}^{\text{slot}}) \quad (19)$$

highest-ranked predicted experts. Their H2D copies are issued asynchronously. When the true router completes, an exact expert that has arrived can execute immediately; an incorrect prefetch is discarded or retained according to RHAC, and an expert that remains unavailable becomes a residual miss. This bound limits the scheduled transfers to the estimated overlap window; inaccurate predictions may nevertheless consume bandwidth and cache capacity.

3.5 Approximate Expert Substitution

Cross-layer prefetching reduces exposed communication but cannot guarantee that every selected expert is available. Synchronously loading every residual miss preserves exact execution but can introduce a sudden latency spike for the affected token. Thus, MoE-CORE provides an optional approximate expert substitution (AES) path based on the score distribution around the Top- k routing boundary. The mechanism uses score proximity to restrict substitution to low-score routing decisions near the selection boundary. Figure 6 illustrates an example of the substitution process.

For token t at layer l , let $r_{l,t,e}$ denote the score of expert e in the router-score vector $\mathbf{r}_{l,t}$. Order all N_l experts such that

$$r_{l,t,(1)} \geq r_{l,t,(2)} \geq \dots \geq r_{l,t,(N_l)}. \quad (20)$$

The selected set is $\mathcal{S}_{l,t} = \{(1), \dots, (k_l)\}$, and $b_{l,t} = r_{l,t,(k_l+1)}$ is the score immediately below the Top- k boundary. Given a threshold hyperparameter $0 \leq \alpha < 1$, MoE-CORE partitions the selected experts into an exact-only high-score set

$$\mathcal{E}_{l,t}^{\text{high}} = \{e \in \mathcal{S}_{l,t} : r_{l,t,e} > (1 + \alpha)b_{l,t}\}, \quad (21)$$

and a replaceable low-score set

$$\mathcal{E}_{l,t}^{\text{low}} = \{e \in \mathcal{S}_{l,t} : b_{l,t} \leq r_{l,t,e} \leq (1 + \alpha)b_{l,t}\}. \quad (22)$$

Every cache miss in $\mathcal{E}_{l,t}^{\text{high}}$ follows the exact H2D path. Among the low-score selected experts, let

$$\mathcal{U}_{l,t} = \mathcal{E}_{l,t}^{\text{low}} \setminus \mathcal{R}_l = \{u_1, \dots, u_U\} \quad (23)$$

be the residual misses that remain unavailable after prefetching. A substitution candidate must be unelected, already resident, and sufficiently close to the lower side of the routing boundary:

$$\begin{aligned} \mathcal{C}_{l,t} &= \{c \in \mathcal{R}_l \setminus \mathcal{S}_{l,t} : (1 - \alpha)b_{l,t} \leq r_{l,t,c} \leq b_{l,t}\} \\ &= \{c_1, \dots, c_C\}. \end{aligned} \quad (24)$$

MoE-CORE sorts both sets by decreasing router score and performs a one-to-one matching:

$$\begin{aligned} r_{l,t,u_1} &\geq \dots \geq r_{l,t,u_U}, \\ r_{l,t,c_1} &\geq \dots \geq r_{l,t,c_C}, \\ u_p &\leftarrow c_p, \quad p = 1, \dots, \min(U, C). \end{aligned} \quad (25)$$

Thus, the highest-scoring resident candidate replaces the highest-ranked eligible miss, the next candidate replaces the next miss, and no candidate is reused. Because all candidates rank below all selected experts, this ordered matching assigns each miss the closest remaining candidate in routing rank. Any high-score miss and any unmatched low-score miss remain on the exact H2D path.

Let $P = \min(U, C)$ and $\mathcal{U}_{l,t}^{\text{sub}} = \{u_1, \dots, u_P\}$ denote the matched misses. For each pair (u_p, c_p) , the candidate expert c_p replaces only the expert function, while retaining the missed expert's original routing coefficient α_{l,t,u_p} . The routed MoE output becomes

$$\begin{aligned} m_{l,t}^{\text{sub}} &= \sum_{e \in \mathcal{S}_{l,t} \setminus \mathcal{U}_{l,t}^{\text{sub}}} \alpha_{l,t,e} E_{l,e}(h_{l,t}) + \\ &\quad \sum_{p=1}^P \alpha_{l,t,u_p} E_{l,c_p}(h_{l,t}). \end{aligned} \quad (26)$$

The candidate's own router score is used only for matching and does not replace the missed expert's routing coefficient. Eq. 26 changes only the matched near-boundary expert computations and removes their synchronous H2D transfers from the token's critical path. The threshold α limits the score range eligible for substitution. Score proximity is a selection heuristic and does not establish equivalence between the original and substitute expert outputs.

3.6 Layer-Wise Expert Staging for Prefill

Prefill processes many tokens together, so the union of their routing decisions can cover most experts in a layer. Motivated by the observation that complete expert loading can become preferable to selective loading as the token count increases (Wang et al., 2025b), MoE-CORE uses the initial per-layer resident-expert count to set the staging threshold. For layer l , let $K_l^{(0)}$ denote the initially configured number of cached experts and let k_l denote the routing fanout (topk). The threshold is computed as

$$N_l^* = \left\lfloor \frac{K_l^{(0)}}{k_l} \right\rfloor. \quad (27)$$

Thus, N_l^* varies with the configured per-layer cache capacity and routing fanout, rather than being a fixed token count. Layers for which the prefill has at least N_l^* tokens use complete layer-wise staging; smaller prefills retain selective per-layer residency.

The physically reserved prefill region contains two buffers, B_0 and B_1 , each dedicated to the routed-expert weights of one MoE layer. Each buffer is sized for the routed-expert footprint of the largest MoE layer, and the prefill region reserves twice this capacity.

For the complete-staging path, MoE-CORE loads the first two MoE layers into B_0 and B_1 during warm-up. After layer l finishes using one buffer, that buffer becomes available for layer $l + 2$. While layer $l + 1$ computes from the other buffer, the transfer engine loads all experts of layer $l + 2$ into the released buffer. The two buffers alternate until the final MoE layer.

Ignoring warm-up and drain costs, the steady-state latency around layer l is approximated by

$$T_l^{\text{steady}} \approx \max(T_l^{\text{compute}}, T_{l+1}^{\text{H2D}}) + T_l^{\text{sync}}. \quad (28)$$

If the current layer's computation is at least as long as the next layer's transfer, the transfer can be fully

hidden after pipeline warm-up. Otherwise, only their difference remains exposed. The initial loading of the first two layers, the final pipeline drain, and all synchronization costs remain part of TTFT.

4 Evaluation

We evaluate end-to-end inference performance, the contributions of the expert-cache components, and compatibility with multi-token prediction (MTP). The main comparisons and cache-component ablations use exact expert execution; AES and MTP are evaluated separately. Unless otherwise specified, Deepseek is the default model.

4.1 Implementation Settings

Datasets and Models. We use GSM8K (Cobbe et al., 2021), AIME 2026 (Dekoninck et al., 2026), HumanEval (Chen et al., 2021), MMLU (Hendrycks et al., 2021), and LongBench v2 (Bai et al., 2025). They cover mathematical reasoning, code generation, knowledge, and long-context tasks. The evaluated models are DeepSeek-V4-Flash-W4A8 (DeepSeek-AI, 2026) and GLM-5.2-W4A8C8 (GLM-5 Team, 2026).

Platforms. Both models are evaluated on an Ascend 950PR NPU platform with 128 GB of physical NPU memory and a 150-GB host-memory limit. Inference is subject to an 84-GB NPU-memory cap.

Baselines. We compare MoE-CORE with two expert-offloading baselines:

- **Load on Demand (LoD).** The baseline allocates static per-layer expert caches only during decode and synchronously loads a selected expert on a cache miss.
- **vLLM Prefetch.** We use the layer-wise prefetch strategy in vLLM (vLLM Project, 2026). The DeepSeek configuration uses two resident layer buffers and two rotating layer buffers; the GLM configuration uses one resident layer and five offloaded layers.

Request Protocol. All configurations in Table 1 use batch size 1. LoD and MoE-CORE allow up to 1K output tokens, whereas vLLM Prefetch is capped at 128 output tokens to bound the runtime of its substantially slower decoding. DeepSeek and GLM use 200 requests per dataset.

Metrics. We report time to first token (TTFT), time per output token (TPOT), end-to-end latency (E2EL), and output throughput in tokens per second (TPS). For a response containing $n > 1$ output

tokens, TPOT is the generation time after the first token divided by $n - 1$. TTFT-P90 and TPOT-P90 are the reported request-level 90th percentiles. Because the output caps differ, E2EL and TPS comparisons are restricted to LoD and MoE-CORE. Exact cache-hit rate measures the availability of the originally selected experts, including prefetched experts ready before execution. For AES-enabled results, effective expert availability additionally counts selected invocations served by resident substitutes. Means computed from the provided layer-rate arrays weight the recorded layers equally.

4.2 Implementation Details

MoE-CORE reserves separate expert-weight regions for prefill staging and decode caches. Of the 84-GB NPU-memory cap, 95% is budgeted for model inference, including model weights, expert caches, KV cache, activations, and inference workspace; the remaining 5% is reserved for other necessary system allocations. The maximum configured prompt-plus-output length is 368K tokens. In the DeepSeek configuration, the uniform decode allocation uses 84 expert slots per layer. The evaluated nonuniform allocation uses 78 slots for most layers and 120 slots for layers 7, 9, 19, 31, and 37. GLM’s larger expert weights limit its single-NPU decode cache to 28 expert slots per layer. Domain profiles initialize the resident experts, and RHAC updates residency once per generated token. Cross-layer prefetching fetches one predicted expert per layer by default. The substitution threshold is $\alpha = 0.1$. AES is disabled in the main comparison and for the cache-component ablations.

4.3 Effectiveness Evaluation

Table 1 reports the measurements on each model. The MoE-CORE runs use exact expert execution.

On DeepSeek, MoE-CORE reduces mean TPOT from 62.1–70.6 ms to 38.0–44.8 ms. Relative to LoD, the reductions are 42.9% on GSM8K, 33.1% on AIME 2026, 27.9% on HumanEval, 35.3% on MMLU, and 42.9% on LongBench v2, corresponding to 1.39 – $1.75\times$ speedups. E2EL improves by 1.36 – $1.79\times$ across the five workloads. TTFT increases by 1.4–4.1% on the first four workloads; on LongBench v2 it falls from 14.06 to 11.88 s, while E2EL falls from 74.08 to 47.08 s.

On GLM, MoE-CORE records lower mean TPOT than LoD on all five workloads, at 206.6–220.5 versus 229.1–244.8 ms. On LongBench v2, its TPOT is 210.6 versus 242.7 ms and its E2EL

Table 1: Inference performance at batch size 1 with output caps of 1K tokens for LoD and MoE-CORE and 128 tokens for vLLM Prefetch. Both models run on Ascend 950PR. LoD uses num_device_layers=1. AES is disabled in both MoE-CORE configurations, and MTP is disabled in the DeepSeek configuration. Lower latency and higher TPS are better; the best measured value for each dataset, model, and metric is underlined.

Dataset	System	DeepSeek-V4-Flash-W4A8						GLM-5.2-W4A8C8					
		TTFT (ms)		TPOT (ms)		E2EL (s)		TTFT (ms)		TPOT (ms)		E2EL (s)	
		Mean	P90	Mean	P90			Mean	P90	Mean	P90		
GSM8K	LoD	3038.4	3094.8	66.5	71.9	37.84	13.57	8760.3	8920.8	233.9	239.2	180.84	4.07
	vLLM Prefetch	<u>1175.1</u>	<u>1177.7</u>	1268.9	1268.9	162.00	0.78	<u>5662.7</u>	<u>5664.3</u>	5941.5	5941.6	760.24	0.17
	MoE-CORE	3082.4	3124.4	<u>38.0</u>	<u>39.2</u>	<u>21.10</u>	<u>22.35</u>	8950.0	9087.0	<u>210.3</u>	<u>217.1</u>	<u>152.94</u>	<u>4.47</u>
AIME 2026	LoD	2948.4	2984.5	62.9	64.2	67.34	15.21	8860.2	8713.3	244.8	249.1	259.27	3.95
	vLLM Prefetch	<u>1177.6</u>	<u>1179.3</u>	1268.9	1269.1	162.00	0.79	<u>5662.1</u>	<u>5664.5</u>	5941.5	5941.6	760.24	0.17
	MoE-CORE	3026.2	3069.2	<u>42.1</u>	<u>46.5</u>	<u>46.09</u>	<u>22.22</u>	9028.6	8843.4	<u>220.5</u>	<u>226.8</u>	<u>234.65</u>	<u>4.36</u>
HumanEval	LoD	2988.4	3025.2	62.1	63.4	36.38	14.78	8408.8	8910.0	235.6	241.4	182.05	4.05
	vLLM Prefetch	<u>1178.6</u>	<u>1181.6</u>	1269.0	1269.0	162.00	0.79	<u>5663.2</u>	<u>5664.7</u>	5941.7	5941.7	760.25	0.17
	MoE-CORE	3112.2	3153.0	<u>44.8</u>	<u>50.0</u>	<u>26.79</u>	<u>19.17</u>	8515.2	9035.5	<u>212.9</u>	<u>219.5</u>	<u>175.80</u>	<u>4.50</u>
MMLU	LoD	2959.7	3019.2	64.1	65.4	61.20	14.81	8889.3	8910.8	229.1	234.9	<u>86.19</u>	3.93
	vLLM Prefetch	<u>1180.7</u>	<u>1184.5</u>	1269.0	1269.0	162.00	0.79	<u>5932.3</u>	<u>5935.2</u>	5941.8	5941.8	760.54	0.17
	MoE-CORE	3044.4	3106.4	<u>41.5</u>	<u>42.4</u>	<u>43.90</u>	<u>22.39</u>	9032.5	9118.5	<u>206.6</u>	<u>209.6</u>	91.43	<u>4.38</u>
LongBench v2	LoD	14056.4	31351.1	70.6	75.4	74.08	11.40	158894.0	290566.0	242.7	249.2	377.77	2.39
	vLLM Prefetch	<u>9749.7</u>	<u>19006.3</u>	1269.1	1269.8	89.00	0.71	110850.0	202840.2	5941.8	5942.0	865.46	0.15
	MoE-CORE	11879.0	24029.0	<u>40.3</u>	<u>45.0</u>	<u>47.08</u>	<u>18.77</u>	<u>101463.4</u>	<u>195094.5</u>	<u>210.6</u>	<u>217.2</u>	<u>278.90</u>	<u>3.01</u>

is 278.90 versus 377.77 s. E2EL is lower on four of the five workloads; MMLU is the exception, increasing from 86.19 to 91.43 s. Across both models, MoE-CORE records lower TPOT than LoD on all ten model-workload pairs and lower E2EL on nine, demonstrating consistent decode-latency gains and favorable end-to-end performance across the evaluated workloads.

Under the stated output caps, the proposed method achieves lower TPOT than vLLM Prefetch on every dataset: 38.0–44.8 versus 1268.9–1269.1 ms on DeepSeek, and 206.6–220.5 versus 5941.5–5941.8 ms on GLM. vLLM Prefetch, in turn, records lower TTFT on all DeepSeek datasets and four GLM datasets. This trade-off reflects the different expert-staging choices. vLLM Prefetch stages complete expert layers and uses four layer buffers in the DeepSeek configuration, favoring prefill relative to MoE-CORE’s two alternating full-layer buffers. During decode, however, the computation of an individual layer cannot fully hide a complete-layer transfer. MoE-CORE instead retains and prefetches selected experts, avoiding most of these transfers and yielding lower TPOT.

4.4 MTP Compatibility and Best Performance

Table 2 reports the MTP depth sweep for DeepSeek-V4-Flash-W4A8 on GSM8K at batch size 2 with AES enabled. Depth 2 achieves the lowest observed TPOT, 21.5 ms. As MTP depth increases

Table 2: MTP depth sweep on DeepSeek-V4-Flash-W4A8 with GSM8K, batch size 2, and AES enabled. Availability is the unweighted mean of 40 layer-wise effective expert-availability rates. Changes are in percentage points (pp) relative to the MTP-off mean of 99.972%.

MTP depth	TPOT (ms)	Accept. (%)	Avail. (%)	Δ (pp)
1	23.1	95.21	99.785	−0.186
2	21.5	78.33	99.607	−0.365
3	22.3	60.78	99.439	−0.532

from 1 to 3, the acceptance rate decreases from 95.21% to 78.33% and 60.78%, while speculative H2D transfers increase. At depth 3, lower acceptance and additional transfer overhead offset the benefit of longer predictions, raising TPOT from 21.5 to 22.3 ms. Thus, depth 2 provides the best measured balance between prediction acceptance and transfer overhead on this platform.

Across MTP depths 1–3, effective expert availability exceeds 99.4%; the largest decrease from the MTP-off reference is 0.532 percentage points. These measurements demonstrate compatibility with MTP and identify depth 2 as the best measured setting. Effective availability includes invocations served by resident substitutes rather than only exact hits of the originally selected experts.

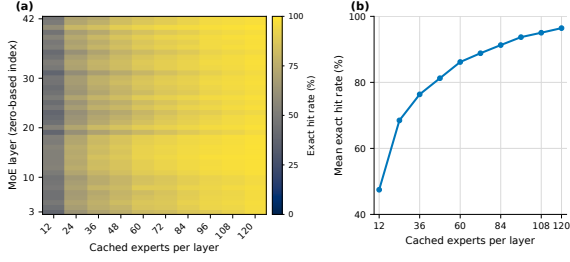


Figure 7: Effect of uniform expert-cache capacity on DeepSeek-V4-Flash-W4A8 with GSM8K. (a) Exact hit rate on each comparable layer (zero-based indices 3–42) for capacities of 12–120 experts per layer. (b) Unweighted mean exact hit rate over those 40 layers.

4.5 Ablation

Unless otherwise stated, the ablations use GSM8K and DeepSeek-V4-Flash. Each experiment changes one component while staying within the same expert-memory budget and retaining the inference configuration.

4.5.1 Effect of Expert-Cache Capacity

Figure 7 evaluates the relationship between cache capacity and exact expert hit rate on the 40 comparable DeepSeek MoE layers (zero-based indices 3–42). Increasing the number of cached experts per layer from 12 to 120 raises the unweighted mean hit rate from 47.47% to 96.43%. Expanding from 12 to 24 experts gains 21.01 percentage points, whereas expanding from 84 to 120 gains only 5.13 points. Thus, the benefit of uniformly increasing cache capacity diminishes as more experts are resident.

4.5.2 Effect of Decode-Cache Components

Figure 8 compares the full configuration with three ablations: removing RHAC, removing domain-conditioned initialization, and replacing the nonuniform allocation with a uniform per-layer allocation. The full configuration’s nonuniform capacity assignment was selected through repeated offline trials. Disabling RHAC increases TPOT from 36.0 to 46.8 ms, corresponding to a 30.0% degradation. The exact expert hit rate simultaneously decreases from above 90% in most layers to approximately 60–82%. Because this reduction occurs throughout the model, RHAC is the principal mechanism for maintaining an effective resident working set during decode.

Removing the domain hot-expert profile increases TPOT to 36.5 ms (1.4%), while replacing

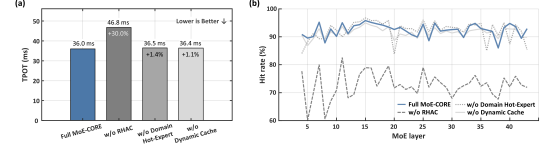


Figure 8: Ablation of the decode expert cache. (a) TPOT for the full system and each ablated configuration. (b) Per-layer exact cache hit rate under the same configurations. The “w/o Dynamic Cache” variant uses uniform per-layer capacity.

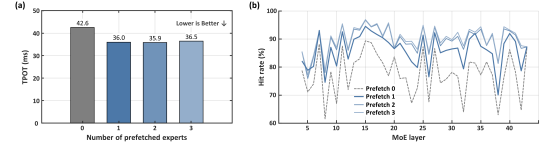


Figure 9: Effect of cross-layer prefetch depth. (a) TPOT for zero to three prefetched experts. (b) Per-layer exact cache hit rate at each depth.

the nonuniform allocation with a uniform per-layer cache increases TPOT to 36.4 ms (1.1%). Their layer-wise curves remain close to that of the full system, but exhibit localized reductions in hit rate. Domain profiles initialize cache contents, the selected nonuniform allocation gives low-hit layers more capacity, and RHAC adapts the resident set during decoding. The complete system combines these mechanisms and achieves the lowest TPOT.

4.5.3 Effect of Cross-Layer Prefetch Depth

Figure 9 compares cross-layer prefetch depths from zero to three experts. Without prefetching, TPOT reaches 42.6 ms. Prefetching one expert reduces TPOT to 36.0 ms, an improvement of 15.5%. Increasing the depth to two experts produces the minimum TPOT of 35.9 ms, only 0.1 ms below the one-expert configuration, whereas a depth of three increases TPOT to 36.5 ms.

Mean exact hit rate rises from 89.05% without prefetching to 92.43%, 93.81%, and 94.05% at depths one, two, and three, respectively. Beyond depth one, the additional hit-rate gains are small. At depth three, the H2D time for three predicted experts cannot be fully hidden by the limited per-layer computation during decode, so the additional transfers outweigh the marginal hit-rate gain and increase TPOT. The first predicted expert thus captures nearly all of the latency benefit. MoE-CORE uses a depth of one by default because it remains within 0.3% of the minimum TPOT while issuing fewer speculative transfers.

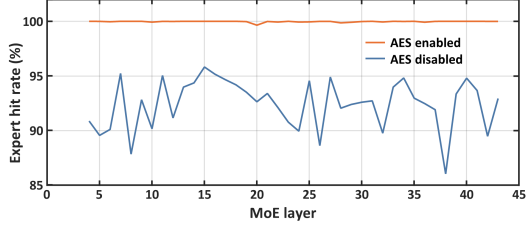


Figure 10: Expert availability with and without AES. The disabled curve measures availability of the original selected experts; the enabled curve additionally counts invocations served by resident substitutes.

Table 3: Recorded AES latency under different decode configurations.

AES	Batch size	MTP depth	TPOT (ms)
Disabled	1	Off	36.0
Enabled	1	Off	37.7
Enabled	2	2	22.0

4.5.4 Effect of Approximate Expert Substitution

Approximate expert substitution (AES) resolves an eligible residual miss with an unselected expert that is already resident on the NPU. We define *effective expert availability* as the fraction of selected expert invocations that can proceed without a demand-triggered H2D transfer. With AES disabled, it reflects availability from caching and completed prefetches; with AES enabled, it also counts invocations served by resident substitutes.

The recorded mean over 40 layer rates rises from 92.43% without AES to 99.972% with AES. Figure 10 visualizes this near-complete effective availability. This near-unity layer mean shows that, on average across the evaluated layers, MoE-CORE serves nearly all selected expert invocations without a demand-triggered H2D transfer.

At batch size 1 with MTP disabled, enabling AES increases TPOT from 36.0 to 37.7 ms, a 4.7% overhead, due to the CPU-side computation required for expert substitution.

5 Related Work

Expert offloading and heterogeneous execution. MoE-Infinity exploits activation traces to guide expert caching and prefetching, while FineMoE uses fine-grained routing patterns and prompt-derived hints for expert management (Xue et al., 2025; Yu et al., 2025). Fiddler and KTransformers use host computation as well as device execution to ac-

commodate experts under limited device memory (Kamahori et al., 2025; Chen et al., 2025). HOB-BIT uses lower-precision versions of less critical missed experts to reduce transfer cost (Tang et al., 2024). MoE-CORE instead focuses on exact-expert residency and transfer scheduling within a single-NPU memory budget.

Transfer scheduling and routing-guided residency. MoE-Lightning pipelines host, device, and I/O work with paged weights, whereas Klot-ski uses an expert-aware multi-batch pipeline to increase computation–transfer overlap (Cao et al., 2024; Fang et al., 2025). DuoServe-MoE explicitly separates prefill and decode scheduling and applies phase-specific expert prefetching (Zhang et al., 2026). SiDA-MoE and ProMoE train auxiliary predictors for proactive expert loading; Fate and AdapMoE exploit cross-layer routing information (Du et al., 2024; Song et al., 2025; Fang et al., 2026b; Zhong et al., 2024a). DoMoE additionally uses domain-specific routing tables to constrain semantic expert prediction (Mu et al., 2026). In contrast to predictor-centered designs, MoE-CORE uses a workload-domain profile to initialize cache contents, updates residency from observed routing history, and evaluates the original next-layer router early for prefetching.

Approximate execution and speculative decoding. SMOE and BuddyMoE examine expert substitution when selected experts are unavailable, while SpecMoEOff uses speculative decoding to enlarge expert workloads and hide offloading latency (Zhu et al., 2026; Wang et al., 2025a,b). MoE-CORE’s optional substitution is restricted to resident, unselected experts close to the selected expert’s routing score at the Top- k boundary; it does not assert functional equivalence from score proximity. Our MTP experiment assesses the compatibility of the expert-offloading path with multi-token prediction rather than introducing a new speculation algorithm. More broadly, long-context serving systems such as Sarathi-Serve and DistServe schedule prefill and decode to improve serving efficiency (Agrawal et al., 2024; Zhong et al., 2024b); our focus is expert-weight residency and transfer within a single appliance.

6 Conclusion

MoE-CORE addresses expert-weight offloading for memory-constrained MoE inference on compact,

single-NPU AI appliances. Measurements across five workloads reveal recurrent low-hit layers under uniform per-layer cache sizes and workload-dependent expert rankings; the examined routing-score distribution also illustrates a narrow gap near the Top- k boundary. These observations motivate a runtime that separates prefill staging from decode-time expert management and coordinates nonuniform cache capacity, domain-informed initialization, routing-history-aware replacement, and cross-layer prefetching. Across five workloads per model and under the stated output caps, the main exact-execution configuration records mean TPOT of 38.0–44.8 ms on DeepSeek and 206.6–220.5 ms on GLM, versus 1268.9–1269.1 and 5941.5–5941.8 ms, respectively, for the evaluated vLLM Prefetch configurations. Under an 84-GB NPU-memory cap, a separate DeepSeek experiment with optional approximate substitution reaches the best measured TPOT of 21.5 ms at MTP depth 2. The results support coordinated residency and transfer scheduling as a practical means of reducing decode latency when the full MoE expert set cannot remain on the device.

Limitations

MoE-CORE does not jointly optimize expert-weight offloading and KV-cache management, and its evaluation is limited to a single NPU; multi-NPU expert-weight placement remains future work. Further study of longer MTP depths may clarify when their additional H2D transfers can be offset by accepted predictions. We also plan to coordinate the currently separate prefill and decode expert caches across consecutive sessions. Experts retained by RHAC after one session may be reused in the next session’s prefill, allowing layer-wise staging to transfer only experts that are not already resident. Finally, RHAC’s CPU-side updates add latency; reducing this computation is another direction for optimization.

References

Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. [Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve](#). In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA. USENIX Association.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2025. [Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks](#). *Preprint*, arXiv:2412.15204.

Robert G. Brown. 1956. Exponential smoothing for predicting demand. Presented at the Tenth National Meeting of the Operations Research Society of America. Also issued as a monograph by Arthur D. Little, Inc.

Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2024. [Moe-lightning: High-throughput moe inference on memory-constrained gpus](#). *Preprint*, arXiv:2411.11217.

Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. 2025. [Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models](#). In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP ’25*, page 1014–1029, New York, NY, USA. Association for Computing Machinery.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. [Evaluating large language models trained on code](#). *Preprint*, arXiv:2107.03374.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. [Training verifiers to solve math word problems](#). *Preprint*, arXiv:2110.14168.

DeepSeek-AI. 2026. [Deepseek-v4: Towards highly efficient million-token context intelligence](#). *Preprint*, arXiv:2606.19348.

Jasper Dekoninck, Nikola Jovanović, Tim Gehringer, Kári Rögnauldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. 2026. [Beyond benchmarks: Matharena as an evaluation platform for mathematics with llms](#). *Preprint*, arXiv:2605.00674.

Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Helen Li, and Yiran Chen. 2024. [Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models](#). In *Proceedings of Machine Learning and Systems*, volume 6, pages 224–238.

- Shicheng Fang, Yuxin Wang, Xiaoran Liu, Jiahao Lu, Chuanyuan Tan, Xinchu Chen, Yining Zheng, Xuanjing Huang, and Xipeng Qiu. 2026a. [Agentlong-bench: A controllable long benchmark for long-contexts agents via environment rollouts](#). *Preprint*, arXiv:2601.20730.
- Zhiyuan Fang, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. 2025. [Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline](#). *Preprint*, arXiv:2502.06888.
- Zhiyuan Fang, Xingfan Yu, Yuegui Huang, Zicong Hong, Yufeng Lyu, Wuhui Chen, Yue Yu, and Fan Yu. 2026b. [Fate: Fast edge inference of mixture-of-experts models via cross-layer gate](#). In *Proceedings of the ACM Web Conference 2026*, WWW '26, page 5537–5546, New York, NY, USA. Association for Computing Machinery.
- GLM-5 Team. 2026. [Glm-5: from vibe coding to agentic engineering](#). *Preprint*, arXiv:2602.15763.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, yelong shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2024. [Tora: A tool-integrated reasoning agent for mathematical problem solving](#). In *International Conference on Learning Representations*, volume 2024, pages 48362–48395.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). *Preprint*, arXiv:2009.03300.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. [Swe-bench: Can language models resolve real-world github issues?](#) In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. 2025. [Fiddler: Cpu-gpu orchestration for fast inference of mixture-of-experts models](#). In *International Conference on Learning Representations*.
- Kimi-Team. 2026. [Kimi k3: Open frontier intelligence](#).
- Yao Mu, Fahao Chen, Wenbin Zhu, Mengying Zhao, Zhaoyan Shen, and Dongxiao Yu. 2026. [Domoe: Domain-aware semantic expert prediction for efficient moe inference under expert offloading](#). In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence*, pages 5901–5909.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. [Outrageously large neural networks: The sparsely-gated mixture-of-experts layer](#). *Preprint*, arXiv:1701.06538.
- Xiaoni Song, Zihang Zhong, Rong Chen, and Haibo Chen. 2025. [Promoe: Fast moe-based llm serving using proactive caching](#). *Preprint*, arXiv:2410.22134.
- Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. [Hobbit: A mixed precision expert offloading system for fast moe inference](#). *Preprint*, arXiv:2411.01433.
- vLLM Project. 2026. [Prefetch-based cpu offloading in vllm](#). Source code in the vLLM repository. Accessed 2026-09-30.
- Yun Wang, Lingyun Yang, Senhao Yu, Yixiao Wang, Ruixing Li, Zhixiang Wei, James Yen, and Zhengwei Qi. 2025a. [Buddymoe: Exploiting expert redundancy to accelerate memory-constrained mixture-of-experts inference](#). *Preprint*, arXiv:2511.10054.
- Zhibin Wang, Zhonghui Zhang, Yuhang Zhou, Zibo Wang, Mo Zhou, Peng Jiang, Weilin Cai, Chengying Huan, Rong Gu, Sheng Zhong, and Chen Tian. 2025b. [Accelerating mixture-of-experts inference by hiding offloading latency with speculative decoding](#). *Preprint*, arXiv:2508.21706.
- Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. 2025. [Moe-infinity: Efficient moe inference on personal machines with sparsity-aware expert cache](#). *Preprint*, arXiv:2401.14361.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. [tau-bench: A benchmark for tool-agent-user interaction in real-world domains](#). *arXiv preprint arXiv:2406.12045*.
- Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2025. [Edgemoe: Empowering sparse large language models on mobile devices](#). *Preprint*, arXiv:2308.14352.
- Hanfei Yu, Xingqi Cui, Hong Zhang, Hao Wang, and Hao Wang. 2025. [Taming latency-memory trade-off in moe-based llm serving via fine-grained expert offloading](#). *Preprint*, arXiv:2502.05370.
- Yuning Zhang, Grant Pinkert, Nan Yang, Yanli Li, and Dong Yuan. 2026. [Duoserve-moe: Dual-phase expert prefetch and caching for llm inference qos assurance](#). *Preprint*, arXiv:2509.07379.
- Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. 2024a. [Adapmoe: Adaptive sensitivity-based expert gating and management for efficient moe inference](#). In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, ICCAD '24*, page 1–9. ACM.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024b. [Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving](#). *Preprint*, arXiv:2401.09670.
- Guoying Zhu, Meng Li, Haipeng Dai, Xuechen Liu, Weijun Wang, Keran Li, Jun xiao, Ligeng Chen, and Wei Wang. 2026. [Smoe: An algorithm-system co-design for pushing moe to the edge via expert substitution](#). *Preprint*, arXiv:2508.18983.